# Производственная практика
# РЭУ, 4 курс

## Оглавление

Производственная практика....1
1. Производственная практика....5
2. Индивидуальное задание на практику....5
Название задания....6
2.1. Цель задания....6
2.2. Задачи студента....6
2.3. Требования к результату выполнения задания....7
3. Содержание индивидуального задания....7
3.1. Функциональные требования....7
3.2. Нефункциональные требования....8
4. Рекомендуемая архитектура модуля....8
4.1. Модель товара....9
4.2. Коллекция в памяти....9
4.3. Сервис импорта....10
4.4. Результат импорта....10
5. Формат CSV-файла....11
Обязательные поля....11
Правила валидации....12
6. Требования к ASP.NET MVC-интерфейсу....12
6.1. Страница импорта товаров....12
6.2. Страница списка товаров....13
7. План выполнения задания....13
1. Организационно-подготовительный этап и анализ требований....13
Задачи....13
Ожидаемый результат....14

2. Проектирование модели данных и репозитория в памяти.....14
Задачи.....14
Ожидаемый результат.....15
Пример минимального кода.....15
3. Реализация чтения CSV-файла и валидации данных.....15
Задачи.....15
Ожидаемый результат.....16
Пример структуры DTO.....16
Пример логики валидации.....16
4. Интеграция модуля импорта в ASP.NET MVC-приложение.....17
Задачи.....17
Ожидаемый результат.....18
Пример контроллера.....18
5. Тестирование, отладка, инспектирование кода и защита задания 19
Задачи.....19
Ожидаемый результат.....19
8. Тестовые сценарии.....19
Сценарий 1. Успешный импорт корректного CSV-файла.....20
Сценарий 2. Файл с неверной ценой.....20
Сценарий 3. Пустой файл.....20
Сценарий 4. Дубликат артикула.....21
Сценарий 5. Отрицательный остаток.....21
Сценарий 6. Файл с кириллицей.....21
9. Требования к коду и стандартам.....22
10. Требования к отчету по практике.....22
10.1. Титульный лист.....22
10.2. Цель и задачи практики.....23

10.3. Описание задания.....23
10.4. Архитектура решения.....23
10.5. Формат CSV-файла.....23
10.6. Реализация функциональности.....24
10.7. Тестирование и отладка.....24
10.8. Выводы.....24
10.9. Приложения.....24
11. Обязательные и дополнительные элементы задания.....25
11.1. Обязательные требования.....25
11.2. Дополнительные элементы.....25
12. Варианты индивидуализации задания.....26
Вариант 1. Электроника.....26
Вариант 2. Книги.....26
Вариант 3. Одежда.....27
13. Рекомендуемые технологии и инструменты.....27
14. Пример минимального рабочего сценария для проверки.....28
15. Чек-лист готовности задания к защите.....28

# 1. Производственная практика

Рабочая программа производственной практики по направлению подготовки специалистов среднего звена:

- специальность **09.02.07 «Информационные системы и программирование»**;
- квалификация — **Программист**;
- программа практики **ПП02.01 «Осуществление интеграции программных модулей»**;

В программе закреплены следующие профессиональные компетенции:

| Код | Содержание компетенции |
|---|---|
| ПК 2.1 | Разрабатывать требования к программным модулям на основе анализа проектной и технической документации |
| ПК 2.2 | Выполнять интеграцию модулей в программное обеспечение |
| ПК 2.3 | Выполнять отладку программного модуля с использованием специализированных средств |
| ПК 2.4 | Разрабатывать тестовые наборы и тестовые сценарии для программного обеспечения |
| ПК 2.5 | Инспектировать компоненты ПО на соответствие стандартам кодирования |

# 2. Индивидуальное задание на практику

Разработать ASP.NET MVC-модуль на C#, предназначенный для наполнения электронного магазина товарами путем чтения данных из CSV-файла и загрузки корректных записей в коллекцию в памяти. Модуль должен обеспечивать:

1. чтение CSV-файла с данными о товарах;
2. проверку обязательных полей и допустимости значений;
3. формирование списка ошибок с указанием номеров строк;
4. добавление корректных товаров в память приложения;
5. отображение результата импорта;

6. просмотр списка загруженных товаров;
7. отладку, тестирование и оформление отчета по практике.

Задание направлено на формирование практических умений по разработке требований к программному модулю, его интеграции в ASP.NET MVC-приложение, отладке, тестированию и проверке качества кода.

## Название задания

**Разработка модуля импорта товаров в электронный магазин из CSV-файлов с хранением данных в коллекциях в памяти.**

Задание выполняется в рамках производственной практики и соответствует компетенциям:

- ПК 2.1 — разработка требований к программному модулю;
- ПК 2.2 — интеграция программного модуля в приложение;
- ПК 2.3 — отладка программного модуля;
- ПК 2.4 — разработка тестовых сценариев и проверка работоспособности;
- ПК 2.5 — инспектирование кода на соответствие стандартам.

---

## 2.1. Цель задания

Сформировать у студента практические умения:

- анализировать требования к программному модулю;
- проектировать простую структуру ASP.NET MVC-приложения;
- работать с файлами CSV в C#;
- использовать коллекции в памяти, например `List<T>`, `Dictionary<TKey,TValue>`;
- реализовывать сервисный слой для импорта данных;
- интегрировать модуль в ASP.NET MVC-приложение;
- выполнять отладку и тестирование кода;
- оформлять техническую документацию по выполненной работе.

---

## 2.2. Задачи студента

В ходе выполнения задания студент должен:

1. Изучить требования к модулю импорта товаров.
2. Подготовить описание формата CSV-файла.
3. Спроектировать модель товара и структуру данных в памяти.

4. Реализовать чтение CSV-файла.
5. Выполнить валидацию читаемых данных.
6. Загрузить корректные данные в коллекцию в памяти.
7. Создать ASP.NET MVC-интерфейс для запуска импорта и просмотра товаров.
8. Выполнить отладку модуля.
9. Подготовить тестовые сценарии.
10. Провести проверку кода на соответствие базовым стандартам.
11. Оформить отчет по практике.

---

## 2.3. Требования к результату выполнения задания

По окончании недели студент должен иметь практический опыт:

- разработки требований к программному модулю;
- проектирования простого ASP.NET MVC-приложения;
- работы с файлами CSV;
- использования коллекций в памяти для хранения данных;
- интеграции модуля импорта в веб-приложение;
- отладки кода средствами IDE;
- разработки тестовых сценариев;
- проверки качества кода.

Студент должен уметь:

- использовать систему контроля версий, например Git;
- применять методы получения кода с заданной функциональностью и степенью качества;
- читать техническую документацию;
- оформлять отчет по выполненному программному модулю.

---

# 3. Содержание индивидуального задания

## 3.1. Функциональные требования

Студент должен разработать ASP.NET MVC-модуль, который позволяет:

1. Загружать CSV-файл с данными о товарах.
2. Читать данные из файла построчно.
3. Преобразовывать строки CSV в объекты C#.
4. Проверять корректность данных.

5. Добавлять корректные товары в коллекцию в памяти.
6. Показывать пользователю результат импорта:
    - количество успешно загруженных товаров;
    - количество ошибок;
    - список ошибок с номерами строк и описанием проблемы.
7. Отображать список товаров, загруженных из CSV-файла.

База данных в задании не требуется. Данные хранятся только в памяти приложения. После перезапуска приложения данные могут быть потеряны — это допустимо.

---

## 3.2. Нефункциональные требования

Код должен соответствовать базовым требованиям качества:

1. Код должен быть читаемым и понятным.
2. Должно соблюдено разделение ответственности:
    - Controller не должен содержать сложную бизнес-логику;
    - логика импорта должна выноситься в отдельный сервис или класс;
    - хранение данных должно быть отделено от представления.
3. Должна быть обработка ошибок:
    - файл не найден;
    - пустой файл;
    - неверные заголовки CSV;
    - неверное значение цены;
    - отрицательный остаток;
    - дубликаты артикула;
    - пустое название товара.
4. Должна быть возможность повторного запуска импорта без падения приложения.
5. Код должен быть размещен в системе контроля версий, например Git.
6. Должен быть подготовлен краткий отчет с описанием решения и результатами тестирования.

---

# 4. Рекомендуемая архитектура модуля

Для выполнения задания рекомендуется использовать следующую структуру:

```
ECommerceCsvImport/
├── Controllers/
│   └── ProductsController.cs
├── Models/
│   ├── Product.cs
│   ├── ImportResult.cs
│   └── ImportError.cs
├── Services/
│   ├── IProductRepository.cs
│   ├── InMemoryProductRepository.cs
│   ├── IProductImportService.cs
│   └── ProductImportService.cs
├── Views/
│   └── Products/
│       ├── Index.cshtml
│       └── Import.cshtml
└── wwwroot/
    └── csv/
        └── sample_products.csv
```

---

## 4.1. Модель товара

Пример класса `Product`:

```csharp
public class Product
{
    public string Sku { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public string Currency { get; set; } = "RUB";
    public string CategoryName { get; set; } = string.Empty;
    public int StockQuantity { get; set; }
    public string ImageUrl { get; set; } = string.Empty;
    public bool IsActive { get; set; }
}
```

---

## 4.2. Коллекция в памяти

Для хранения товаров можно использовать:

```csharp
public class InMemoryProductRepository : IProductRepository
{
    private readonly List<Product> _products = new();

    public void Add(Product product)
    {
```

```
        _products.Add(product);
    }

    public IReadOnlyList<Product> GetAll()
    {
        return _products;
    }

    public Product? GetBySku(string sku)
    {
        return _products.FirstOrDefault(p => p.Sku == sku);
    }
}
```

Допускается также использование:

- `Dictionary<string, Product>` — если ключом является артикул товара;
- `List<Product>` — для простого перечисления товаров.

---

## 4.3. Сервис импорта

Пример интерфейса:

```
public interface IProductImportService
{
    ImportResult ImportFromCsv(string filePath);
}
```

Класс `ProductImportService` должен выполнять:

1. Проверку существования файла.
2. Чтение CSV-файла.
3. Разбор строк.
4. Валидацию данных.
5. Создание объектов `Product`.
6. Добавление товаров в репозиторий.
7. Формирование результата импорта.

---

## 4.4. Результат импорта

Пример класса:

```
public class ImportResult
{
    public int TotalRows { get; set; }
    public int ImportedCount { get; set; }
```

```
    public int ErrorCount { get; set; }
    public List<ImportError> Errors { get; set; } = new();
}

public class ImportError
{
    public int RowNumber { get; set; }
    public string Message { get; set; } = string.Empty;
}
```

---

# 5. Формат CSV-файла

Студент должен использовать CSV-файл с заголовками.

Пример:

```
sku,name,description,price,currency,category_name,stock,image_url,is_a
ctive
1001,Ноутбук Lenovo IdeaPad 3,Бюджетный ноутбук для работы и
учебы,45990,RUB,Компьютеры,/images/laptop.jpg,true
1002,Мышь Logitech M185,Беспроводная
мышь,1290,RUB,Периферия,/images/mouse.jpg,true
1003,Клавиатура A4Tech,Механическая
клавиатура,3490,RUB,Периферия,/images/keyboard.jpg,false
```

## Обязательные поля

| Поле | Описание | Требование |
|---|---|---|
| `sku` | Артикул товара | Обязательно, уникальное значение |
| `name` | Название товара | Обязательно, не пустое |
| `description` | Описание | Может быть пустым |
| `price` | Цена | Обязательно, число больше 0 |
| `currency` | Валюта | По умолчанию RUB |
| `category_name` | Категория | Обязательно или по умолчанию «Без категории» |
| `stock` | Остаток на складе | Целое число, не меньше 0 |
| `image_url` | Ссылка на изображение | Может быть пустым |
| `is_active` | Активность товара | true/false |

## Правила валидации

Студент должен реализовать проверку:

1. Файл существует и доступен для чтения.
2. Первая строка содержит заголовки.
3. Поле `sku` не пустое.
4. Поле `name` не пустое.
5. Поле `price` является числом и больше 0.
6. Поле `stock` является целым числом и не меньше 0.
7. Поле `is_active` имеет значение `true`, `false`, `1`, `0`.
8. Если артикул уже существует в памяти, студент должен выбрать одно поведение:
    - пропустить дубликат;
    - обновить существующий товар;
    - добавить как новый с предупреждением.

Рекомендуется реализовать вариант **«пропускать дубликаты и добавлять ошибку/предупреждение»**.

---

# 6. Требования к ASP.NET MVC-интерфейсу

Студент должен создать минимум два представления:

## 6.1. Страница импорта товаров

На странице должна быть форма, позволяющая:

- указать путь к CSV-файлу;
- либо загрузить файл через браузер;
- запустить импорт;
- увидеть результат импорта.

Пример полей формы:

```
Путь к файлу: [_____________________________]
[Импортировать товары]
```

Или:

```
Выберите CSV-файл: [Choose file]
[Загрузить и импортировать]
```

Результат должен отображаться в виде таблицы:

| Показатель | Значение |
|---|---|
| Всего строк | 10 |
| Успешно импортировано | 8 |
| Ошибки | 2 |

Список ошибок:

| Строка | Описание ошибки |
|---|---|
| 4 | Поле price должно быть числом больше 0 |
| 7 | Артикул товара уже существует |

## 6.2. Страница списка товаров

После импорта пользователь должен видеть список товаров из памяти:

| Артикул | Название | Цена | Категория | Остаток | Активен |
|---|---|---|---|---|---|
| 1001 | Ноутбук Lenovo IdeaPad 3 | 45990 RUB | Компьютеры | 5 | Да |

Дополнительно можно реализовать:

- поиск по названию;
- фильтрацию по категории;
- сортировку по цене.

Эти функции являются необязательными, но могут быть учтены как дополнительные.

---

# 7. План выполнения задания

---

## 1. Организационно-подготовительный этап и анализ требований

### Задачи

1. Провести инструктаж по технике безопасности и правилам работы на предприятии/в лаборатории.

2. Ознакомиться с целями и задачами практики.
3. Изучить индивидуальное задание.
4. Определить требования к модулю импорта товаров.
5. Подготовить рабочее окружение:
    - Visual Studio или Visual Studio Code;
    - .NET SDK / .NET Framework;
    - Git;
    - браузер для тестирования ASP.NET MVC-приложения.
6. Создать репозиторий в Git.
7. Создать проект ASP.NET MVC.
8. Подготовить пример CSV-файла с тестовыми товарами.

### Ожидаемый результат

Студент должен иметь:

- созданное ASP.NET MVC-приложение;
- репозиторий Git;
- начальный коммит;
- файл `README.md` с описанием задания;
- пример CSV-файла;
- список требований к модулю.

---

## 2. Проектирование модели данных и репозитория в памяти

### Задачи

1. Спроектировать модель товара `Product`.
2. Определить структуру результата импорта `ImportResult`.
3. Создать интерфейс репозитория `IProductRepository`.
4. Реализовать класс `InMemoryProductRepository`.
5. Добавить методы:
    - `Add(Product product);`
    - `GetAll();`
    - `GetBySku(string sku).`
6. Подготовить начальные unit-тесты для репозитория, если используется тестовый фреймворк.
7. Выполнить первый коммит с описанием:

```
Add product model and in-memory repository
```

### Ожидаемый результат

Студент должен иметь:

- класс `Product`;
- интерфейс и реализацию репозитория;
- возможность добавлять товар в память;
- возможность получать список товаров из памяти.

### Пример минимального кода

```
public interface IProductRepository
{
    void Add(Product product);
    IReadOnlyList<Product> GetAll();
    Product? GetBySku(string sku);
}

public class InMemoryProductRepository : IProductRepository
{
    private readonly List<Product> _products = new();

    public void Add(Product product)
    {
        if (product == null)
            throw new ArgumentNullException(nameof(product));

        _products.Add(product);
    }

    public IReadOnlyList<Product> GetAll()
    {
        return _products;
    }

    public Product? GetBySku(string sku)
    {
        return _products.FirstOrDefault(p => p.Sku == sku);
    }
}
```

---

## 3. Реализация чтения CSV-файла и валидации данных

### Задачи

1. Реализовать чтение CSV-файла из локального пути.
2. Разобрать строки CSV на поля.

3. Создать DTO или промежуточную структуру для читаемых данных, например `ProductImportRow`.
4. Реализовать валидацию:
    - пустые обязательные поля;
    - неверный формат цены;
    - отрицательный остаток;
    - некорректное значение `is_active`;
    - дубликаты SKU.
5. Сформировать список ошибок с номерами строк.
6. Добавить корректные товары в репозиторий.
7. Подготовить тестовые CSV-файлы:
    - `valid_products.csv`;
    - `invalid_price.csv`;
    - `duplicate_sku.csv`;
    - `empty_file.csv`.

## Ожидаемый результат

Студент должен иметь работающий сервис импорта, который:

- читает CSV;
- проверяет данные;
- добавляет корректные товары в память;
- возвращает объект `ImportResult` с количеством успешных записей и ошибками.

## Пример структуры DTO

```
public class ProductImportRow
{
    public string Sku { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public string Currency { get; set; } = "RUB";
    public string CategoryName { get; set; } = string.Empty;
    public int StockQuantity { get; set; }
    public string ImageUrl { get; set; } = string.Empty;
    public bool IsActive { get; set; }
}
```

## Пример логики валидации

```
if (string.IsNullOrWhiteSpace(row.Sku))
{
    errors.Add(new ImportError
    {
```

```
            RowNumber = rowNumber,
            Message = "Поле sku не может быть пустым"
        });
        return null;
    }

    if (row.Price <= 0)
    {
        errors.Add(new ImportError
        {
            RowNumber = rowNumber,
            Message = "Цена должна быть больше нуля"
        });
        return null;
    }
```

---

## 4. Интеграция модуля импорта в ASP.NET MVC-приложение

### Задачи

1. Создать `ProductsController`.
2. Реализовать действия контроллера:
   - `Index()` — список товаров;
   - `Import()` GET — форма импорта;
   - `ImportPost()` POST — обработка формы и запуск импорта.
3. Подключить сервис импорта в контроллер через конструктор или фабрику зависимостей.
4. Создать представления:
   - `Views/Products/Index.cshtml`;
   - `Views/Products/Import.cshtml`.
5. Вывести на страницу результат импорта.
6. Реализовать отображение списка товаров после импорта.
7. Выполнить отладку приложения:
   - проверить работу формы;
   - проверить загрузку файла;
   - проверить вывод ошибок;
   - проверить список товаров.
8. Выполнить коммиты с понятными сообщениями, например:

```
Add products controller and import view
Integrate csv import service into mvc application
Fix duplicate sku validation error
```

## Ожидаемый результат

Студент должен иметь работающее веб-приложение, в котором можно:

1. Открыть страницу импорта.
2. Указать путь к CSV-файлу или загрузить файл.
3. Запустить импорт.
4. Увидеть количество успешно загруженных товаров и ошибки.
5. Перейти на страницу списка товаров.
6. Посмотреть товары, загруженные из CSV в память.

## Пример контроллера

```
public class ProductsController : Controller
{
    private readonly IProductRepository _repository;
    private readonly IProductImportService _importService;

    public ProductsController(
        IProductRepository repository,
        IProductImportService importService)
    {
        _repository = repository;
        _importService = importService;
    }

    public ActionResult Index()
    {
        var products = _repository.GetAll();
        return View(products);
    }

    [HttpGet]
    public ActionResult Import()
    {
        return View();
    }

    [HttpPost]
    public ActionResult Import(string filePath)
    {
        if (string.IsNullOrWhiteSpace(filePath))
        {
            ViewBag.Error = "Укажите путь к CSV-файлу";
            return View("Import");
        }

        var result = _importService.ImportFromCsv(filePath);
```

```
        ViewBag.Result = result;
        return View("Import", result);
    }
}
```

---

## 5. Тестирование, отладка, инспектирование кода и защита задания

### Задачи

1. Подготовить тестовые сценарии.
2. Провести функциональное тестирование модуля.
3. Проверить обработку ошибок.
4. Выполнить отладку найденных проблем.
5. Провести самопроверку кода:
    - имена классов и методов;
    - отсутствие «магических чисел»;
    - обработка исключений;
    - читаемость кода;
    - разделение ответственности.
6. Оформить отчет по практике.
7. Подготовить краткую презентацию или устное описание решения.
8. Провести защиту задания перед руководителем практики.

### Ожидаемый результат

Студент должен иметь:

- работающий ASP.NET MVC-модуль импорта товаров из CSV;
- хранение товаров в памяти;
- страницу импорта;
- страницу списка товаров;
- обработку ошибок;
- тестовые сценарии;
- Git-репозиторий с историей коммитов;
- отчет по практике.

---

# 8. Тестовые сценарии

Студент должен проверить модуль на следующих сценариях.

## Сценарий 1. Успешный импорт корректного CSV-файла

**Входные данные:**

```
sku,name,description,price,currency,category_name,stock,image_url,is_a
ctive
1001,Ноутбук Lenovo IdeaPad 3,Бюджетный
ноутбук,45990,RUB,Компьютеры,5,/images/laptop.jpg,true
1002,Мышь Logitech M185,Беспроводная
мышь,1290,RUB,Периферия,10,/images/mouse.jpg,true
```

**Ожидаемый результат:**

- 2 товара успешно импортированы;
- ошибок нет;
- товары отображаются на странице списка.

---

## Сценарий 2. Файл с неверной ценой

**Входные данные:**

```
sku,name,description,price,currency,category_name,stock,image_url,is_a
ctive
1003,Клавиатура A4Tech,Механическая клавиатура,-
500,RUB,Периферия,3,/images/keyboard.jpg,true
```

**Ожидаемый результат:**

- товар не добавлен;
- в результате импорта указана ошибка по строке 2;
- сообщение: «Цена должна быть больше нуля».

---

## Сценарий 3. Пустой файл

**Входные данные:**

пустой CSV-файл.

**Ожидаемый результат:**

- приложение не падает;
- количество импортированных товаров — 0;
- выводится сообщение: «Файл пуст» или «Данные для импорта отсутствуют».

---

## Сценарий 4. Дубликат артикула

**Входные данные:**

```
sku,name,description,price,currency,category_name,stock,image_url,is_a
ctive
1001,Ноутбук Lenovo IdeaPad 3,Бюджетный
ноутбук,45990,RUB,Компьютеры,5,/images/laptop.jpg,true
1001,Ноутбук ASUS VivoBook,Другой
ноутбук,52990,RUB,Компьютеры,7,/images/asus.jpg,true
```

**Ожидаемый результат:**

- первый товар добавлен;
- второй пропущен или обновляет первый в зависимости от выбранной логики;
- в результате импорта указано предупреждение или ошибка о дубликате SKU.

---

## Сценарий 5. Отрицательный остаток

**Входные данные:**

```
sku,name,description,price,currency,category_name,stock,image_url,is_a
ctive
1004,Монитор Dell,Игровой монитор,29990,RUB,Компьютеры,-
5,/images/monitor.jpg,true
```

**Ожидаемый результат:**

- товар не добавлен;
- ошибка: «Остаток на складе не может быть отрицательным».

---

## Сценарий 6. Файл с кириллицей

**Входные данные:**

CSV-файл в кодировке UTF-8 или Windows-1251 с русскими названиями товаров.

**Ожидаемый результат:**

- названия товаров отображаются корректно;
- нет «кракозябр» в браузере.

---

# 9. Требования к коду и стандартам

Студент должен выполнить базовую самопроверку кода по следующим пунктам:

| Критерий | Требование |
|---|---|
| Имена классов | Должны быть понятными, например `ProductImportService`, а не `Class1` |
| Имена методов | Должны отражать действие, например `ImportFromCsv`, `ValidateRow` |
| Обработка ошибок | Нельзя допускать падения приложения из-за неверного CSV |
| Разделение ответственности | Controller не должен сам читать файл и валидировать данные |
| Читаемость | Код должен быть отформатирован, без лишних пустых строк и мусора |
| Комментарии | Ключевые методы должны иметь краткие комментарии |
| Исключения | Должны перехватываться там, где это уместно |
| Магические числа | Цифры вроде `100`, `255` лучше выносить в константы или настройки |
| Git-коммиты | Сообщения коммитов должны быть осмысленными |

# 10. Требования к отчету по практике

Отчет должен содержать следующие разделы:

## 10.1. Титульный лист

Указать:

- наименование учебного заведения;
- специальность;
- квалификацию;
- название практики;
- тему индивидуального задания;

- ФИО студента;
- группу;
- руководителя практики от предприятия/учебного заведения;
- дату выполнения.

## 10.2. Цель и задачи практики

Кратко описать:

- цель производственной практики;
- задачи, которые решал студент в рамках индивидуального задания.

## 10.3. Описание задания

Необходимо указать:

- назначение модуля импорта товаров;
- требования к CSV-файлу;
- требования к функциональности;
- ограничения реализации.

## 10.4. Архитектура решения

Описать:

- структуру проекта;
- роль модели `Product`;
- роль репозитория в памяти;
- роль сервиса импорта;
- роль контроллера и представлений ASP.NET MVC.

Можно привести схему:

```
CSV file
   ↓
ProductImportService
   ↓
Validation + Mapping to Product
   ↓
InMemoryProductRepository
   ↓
ProductsController
   ↓
ASP.NET MVC Views
```

## 10.5. Формат CSV-файла

Привести пример файла и таблицу полей:

| Поле | Тип данных | Обязательность | Описание |
|---|---|---|---|
| sku | string | обязательно | артикул товара |
| name | string | обязательно | название товара |
| price | decimal | обязательно | цена товара |
| stock | int | обязательно | остаток на складе |

## 10.6. Реализация функциональности

Описать:

- как читается CSV;
- как выполняются проверки;
- как товары добавляются в память;
- как результат импорта передается во представление.

Можно привести фрагменты кода, но не весь код целиком.

## 10.7. Тестирование и отладка

Привести таблицу тестовых сценариев:

| № | Сценарий | Входные данные | Ожидаемый результат | Фактический результат |
|---|---|---|---|---|
| 1 | Успешный импорт | valid_products.csv | 2 товара добавлены | Выполнено |
| 2 | Неверная цена | invalid_price.csv | Ошибка по строке | Выполнено |

## 10.8. Выводы

Студент должен указать:

- какие умения были получены;
- какие трудности возникли;
- как они были решены;
- что можно улучшить в дальнейшем, например добавить базу данных, асинхронный импорт, пагинацию, поиск и фильтрацию.

## 10.9. Приложения

В приложения могут входить:

- пример CSV-файла;
- скриншоты страниц;
- список ошибок импорта;
- фрагменты кода;
- история Git-коммитов.

---

# 11. Обязательные и дополнительные элементы задания

## 11.1. Обязательные требования

Для получения положительной оценки студент должен реализовать:

1. ASP.NET MVC-приложение или модуль в существующем приложении.
2. Модель товара `Product`.
3. Хранение товаров в памяти, например `List<Product>`.
4. Чтение CSV-файла.
5. Валидацию данных.
6. Страницу запуска импорта.
7. Страницу просмотра списка товаров.
8. Обработку ошибок.
9. Git-репозиторий с кодом.
10. Отчет по практике.

## 11.2. Дополнительные элементы

Дополнительные функции могут повысить оценку:

1. Загрузка файла через браузер, а не только указание пути.
2. Поиск товаров по названию.
3. Фильтрация по категории.
4. Сортировка по цене или остатку.
5. Страница деталей товара.
6. Подсчет общего количества товаров и общей стоимости.
7. Экспорт списка товаров в CSV.
8. Использование unit-тестов.
9. Логирование ошибок в файл.
10. Поддержка нескольких разделителей CSV: `;`, `,`, `\t`.

---

# 12. Варианты индивидуализации задания

Чтобы задание было индивидуальным, преподаватель может назначить студенту один из вариантов предметной области. Базовая логика импорта остается одинаковой, но меняются дополнительные поля товара и правила валидации.

## Вариант 1. Электроника

Дополнительные поля:

| Поле | Описание | Валидация |
|---|---|---|
| `brand` | Бренд | обязательно |
| `warranty_months` | Гарантия, месяцев | целое число от 0 до 60 |
| `model_number` | Модель | может быть пустым |

Пример CSV:

```
sku,name,description,price,currency,category_name,stock,image_url,is_active,brand,warranty_months,model_number
1001,Ноутбук Lenovo IdeaPad 3,Бюджетный ноутбук,45990,RUB,Компьютеры,5,/images/laptop.jpg,true,Lenovo,24,IdeaPad3-15
```

---

## Вариант 2. Книги

Дополнительные поля:

| Поле | Описание | Валидация |
|---|---|---|
| `isbn` | ISBN | обязательно, 10 или 13 символов |
| `author` | Автор | обязательно |
| `publisher` | Издательство | может быть пустым |
| `publication_year` | Год издания | от 1900 до текущего года + 1 |

Пример CSV:

```
sku,name,description,price,currency,category_name,stock,image_url,is_active,isbn,author,publisher,publication_year
2001,C# для начинающих,Учебник по
```

```
C#,1890,RUB,Книги,12,/images/book.jpg,true,9785446243210,Иванов
И.И.,Питер,2023
```

---

## Вариант 3. Одежда

Дополнительные поля:

| Поле | Описание | Валидация |
|---|---|---|
| `size` | Размер | S, M, L, XL или числовой размер |
| `color` | Цвет | обязательно |
| `material` | Материал | может быть пустым |

Пример CSV:

```
sku,name,description,price,currency,category_name,stock,image_url,is_a
ctive,size,color,material
3001,Футболка белая,Летняя
футболка,990,RUB,Одежда,25,/images/tshirt.jpg,true,M,belii,koton
```

---

# 13. Рекомендуемые технологии и инструменты

| Направление | Рекомендация |
|---|---|
| Язык программирования | C# |
| Фреймворк | ASP.NET MVC, .NET Framework или .NET Core / .NET 8+ |
| IDE | Visual Studio, Visual Studio Code |
| Система контроля версий | Git |
| Хостинг кода | GitHub |
| Браузер | Chrome, Edge, Firefox |
| Файлы данных | CSV в кодировке UTF-8 |
| Тестирование | Ручное тестирование + unit-тесты |

# 14. Пример минимального рабочего сценария для проверки

Студент должен продемонстрировать следующую последовательность:

1. Открыть приложение в браузере.
2. Перейти на страницу импорта товаров.
3. Указать путь к файлу `sample_products.csv`.
4. Нажать кнопку «Импортировать».
5. Получить результат:
   - всего строк;
   - успешно импортировано;
   - ошибки.
6. Перейти на страницу списка товаров.
7. Убедиться, что товары из CSV отображаются в таблице.
8. Повторно запустить импорт с файлом, содержащим ошибку.
9. Показать, что приложение не падает и выводит понятное сообщение об ошибке.

---

# 15. Чек-лист готовности задания к защите

Перед защитой студент должен убедиться, что выполнены следующие пункты:

- ☐ Создано ASP.NET MVC-приложение.
- ☐ Реализован класс `Product`.
- ☐ Реализовано хранение товаров в памяти.
- ☐ Реализовано чтение CSV-файла.
- ☐ Реализована валидация данных.
- ☐ Реализована обработка ошибок.
- ☐ Создана страница импорта.
- ☐ Создана страница списка товаров.
- ☐ Модуль успешно импортирует корректный CSV-файл.
- ☐ Модуль корректно обрабатывает некорректный CSV-файл.
- ☐ Код размещен в Git.
- ☐ Выполнена самопроверка качества кода.
- ☐ Подготовлены тестовые сценарии.
- ☐ Оформлен отчет по практике.
- ☐ Студент может объяснить архитектуру и ключевые фрагменты кода.